

Agenda

- Class 15 – October 29th (10:30 – 12:30)

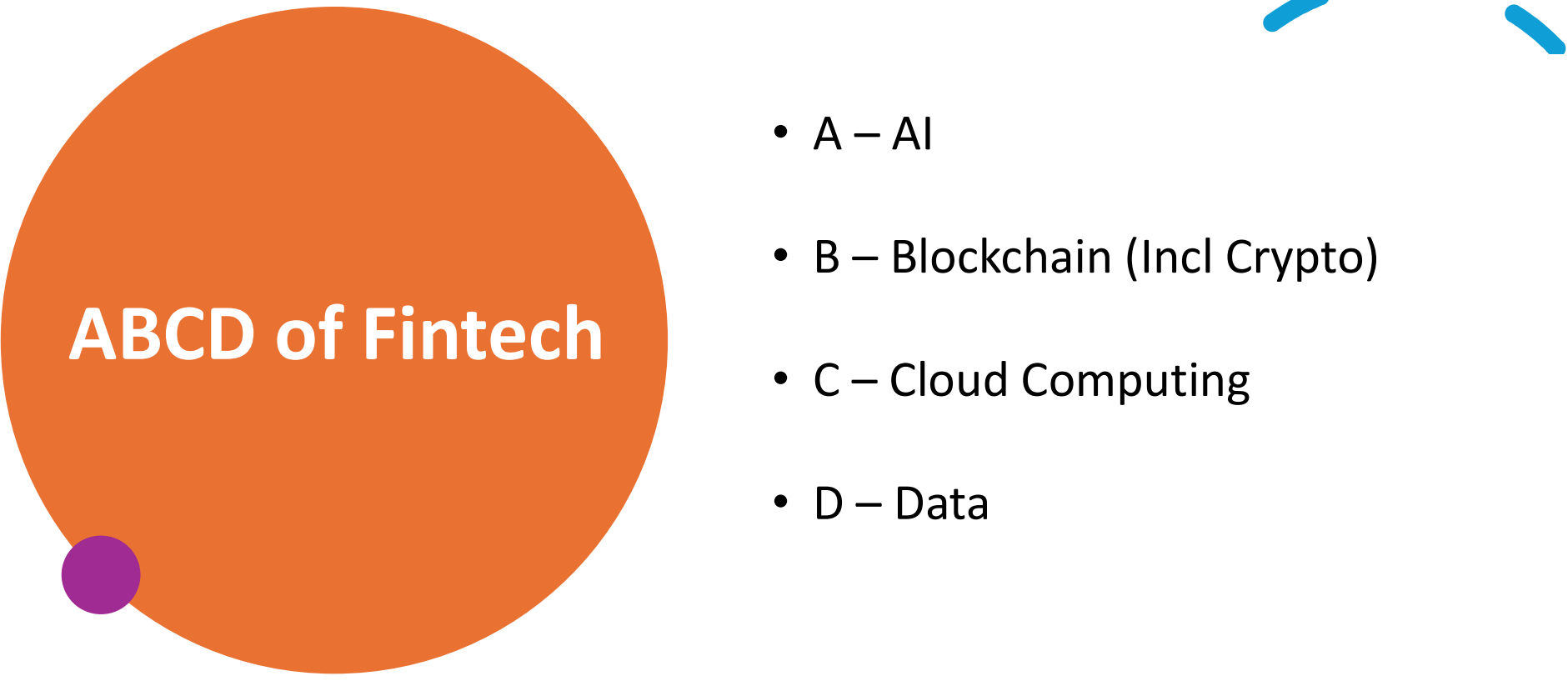
- Starter Quiz
- Scalability, Performance and Resiliency Architecture
- Thought Paper Presentation
 - Embedded Finance – Concept, Evolution, Future Trends (Team 8)
- Scalability, Performance and Resiliency Architecture (Cont'd)

Note:

- Mobile Phones – Do not bring to class or Completely Switch off
- If students have any specific topic they want to discuss, share the topic and we can try to schedule
- Please be seated by your teams

Starter Quiz

- What is new “ABCD” of Fintech



ABCD of Fintech

- A – AI
- B – Blockchain (Incl Crypto)
- C – Cloud Computing
- D – Data

Starter Quiz

- We live in a “VUCA” world – What is VUCA



VUCA

- V – Volatility
 - U – Uncertainty
 - C – Complexity
 - A – Ambiguity
- 

Core Banking Platform Engineering – Scalability, Performance and Resiliency Architecture

Scalability, Performance and Resiliency Architecture – Topics Covered

- **Scalability**

- Scalability Methods
- Processing Models (Auto) Scaling – Design Principles
 - Realtime Processing (API)
 - Near Realtime Processing (Events)
 - Bulk Processing (Batch)
- Database Scaling

- **Performance**

- Key Design Principles
 - Caching
 - Database Indexes
 - Processing Optimization Techniques
- Performance Benchmarks

- **Resiliency**

- Application Resiliency
- Database Resiliency
- Blue Green Deployment Model

Note: Infrastructure Resiliency like Network Resiliency, Hardware Resiliency is not covered in this course

Scalability, Performance and Resiliency Architecture – In the Context of Core Banking Platform Engineering

	Processing Models			Scalability	Performance	Resiliency
Processing Tier	Real time	Near Realtime	Bulk (Batch) Processing			
	Database Types			Scalability	Performance	Resiliency
Persistence (Database) Tier	Transactional Database	Analytical Database				

Scalability

Scalability Basics

Definition of Scalability

Scalability is a system's ability to smoothly manage increased workloads as demand rises (Data Volumes or Transaction Volumes), ensuring an uninterrupted operations and consistent user experience.

Why Scalability

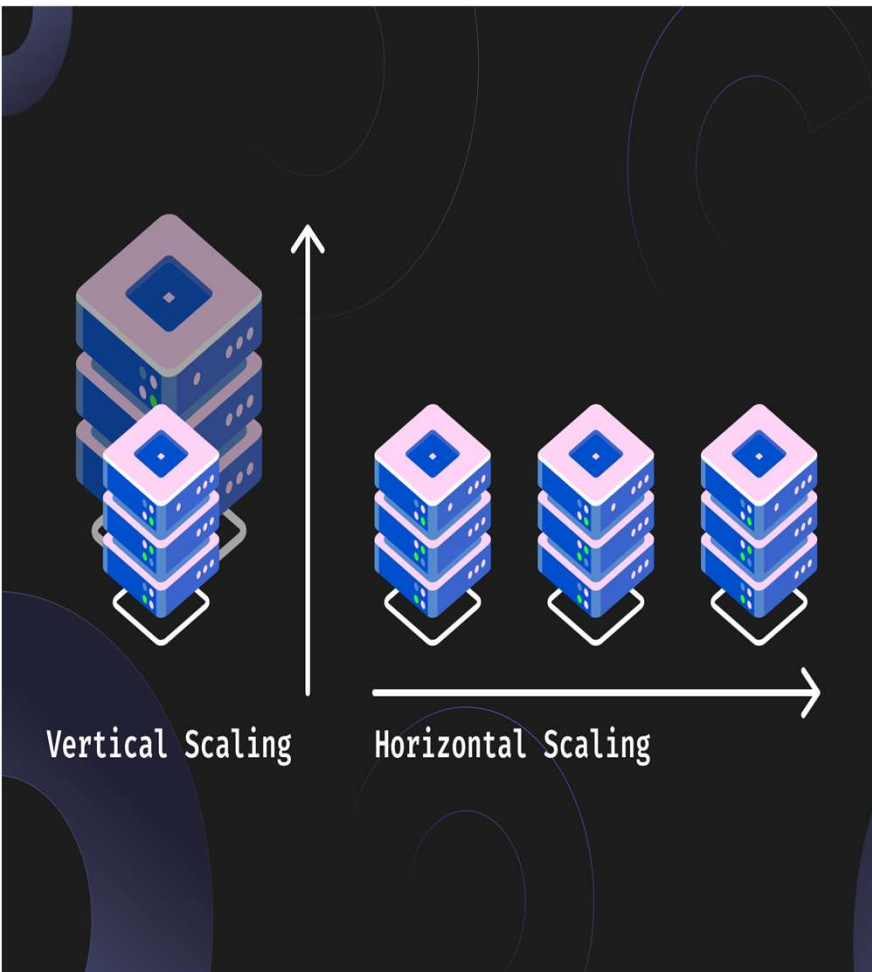
Effective scalability delivers consistent system performance, cost-efficiency, and infrastructure ready for future growth.

Scalability Methods

- Vertical scaling
- Horizontal Scaling

Scalability Measures (In the Context of Core Banking)

- Account Volume: e.g. 200 Million Accounts
- Transaction Volume and Velocity: e.g.
 - Average of 400 Million Transactions a Day
 - Peak TPS of 10,000 TPS



Vertical Scaling

Increasing Server Resources

Vertical scalability means boosting a server's power by adding more CPU, RAM, or storage for improved performance.

Easy Implementation

Vertical scaling is straightforward and often requires no changes to application code, making upgrades simple and quick.

Physical and Cost Limits

Vertical scalability faces physical limits and can become expensive as hardware upgrades reach higher capacities.





Horizontal Scaling

Expanding with More Nodes

Horizontal scalability means adding nodes to grow a system, instead of upgrading existing equipment.

Boosting Performance and Resilience

Adding nodes improves system performance and fault tolerance, supporting higher user and data loads reliably.

Enabling Cost-Effective Growth

Horizontal scaling allows easier, cost-effective expansion to meet changing application needs as demand grows.

Vertical vs Horizontal Scaling Summary

Vertical Scaling

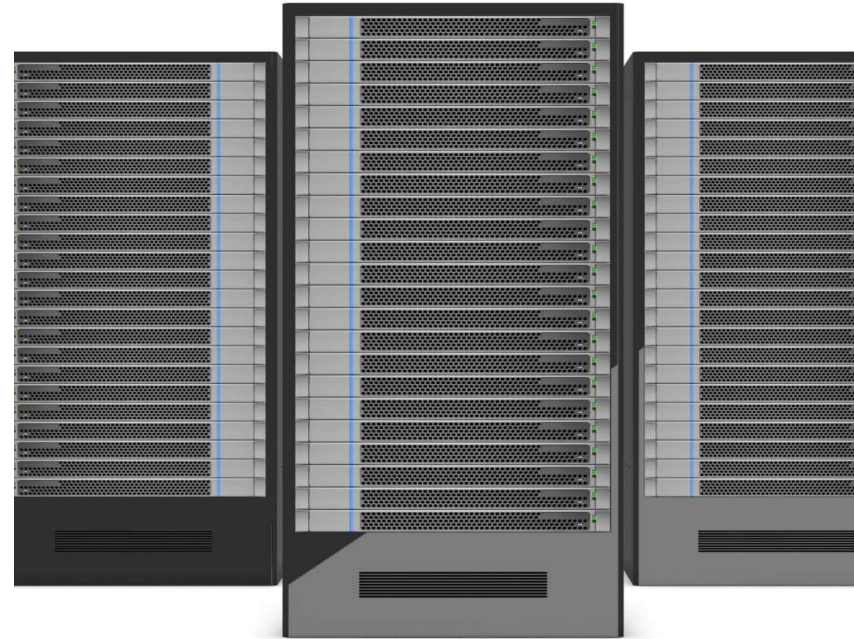
Vertical scaling boosts a single server's performance by adding more resources like CPU or RAM, but has hardware limits.

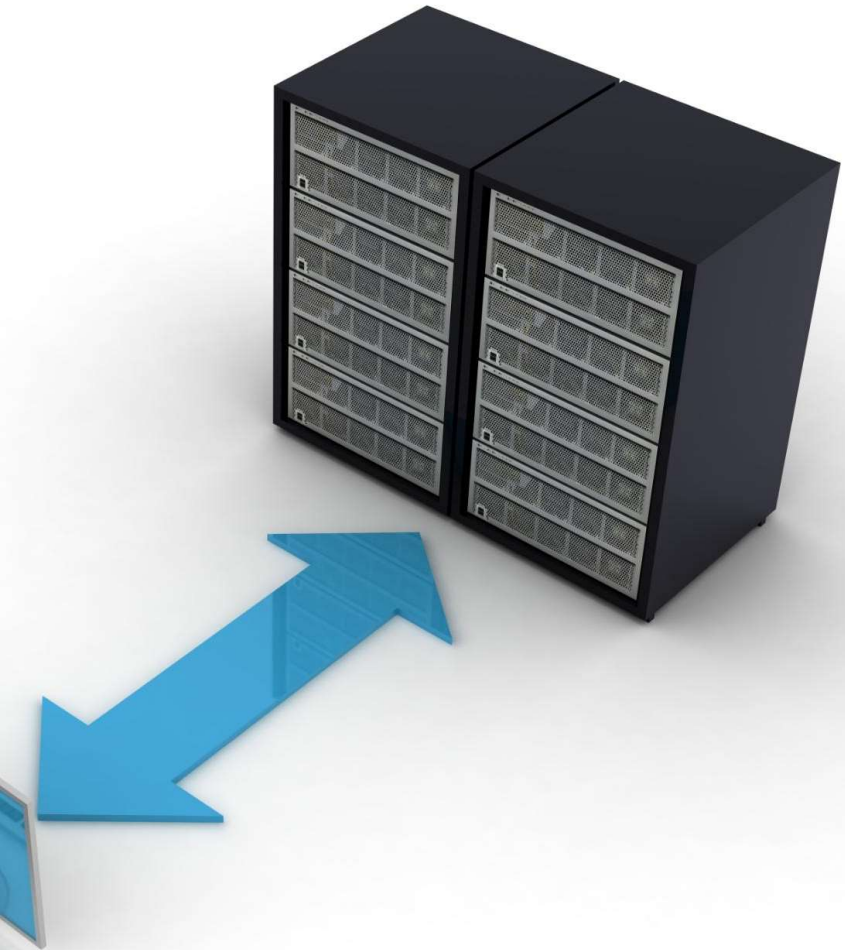
Horizontal Scaling

Horizontal scaling distributes workloads by adding more servers, improving fault tolerance and scalability for large demands.

Flexibility and Implementation

Vertical scaling is simpler but limited, while horizontal scaling offers flexibility for managing growing workloads. Modern applications are generally designed to be able to support Horizontal Scaling





Realtime Processing Model (API) – Scaling Design Techniques

Horizontal Scaling

Partition workloads and distribute data to achieve efficient horizontal scaling for Realtime processing systems.

Microservices

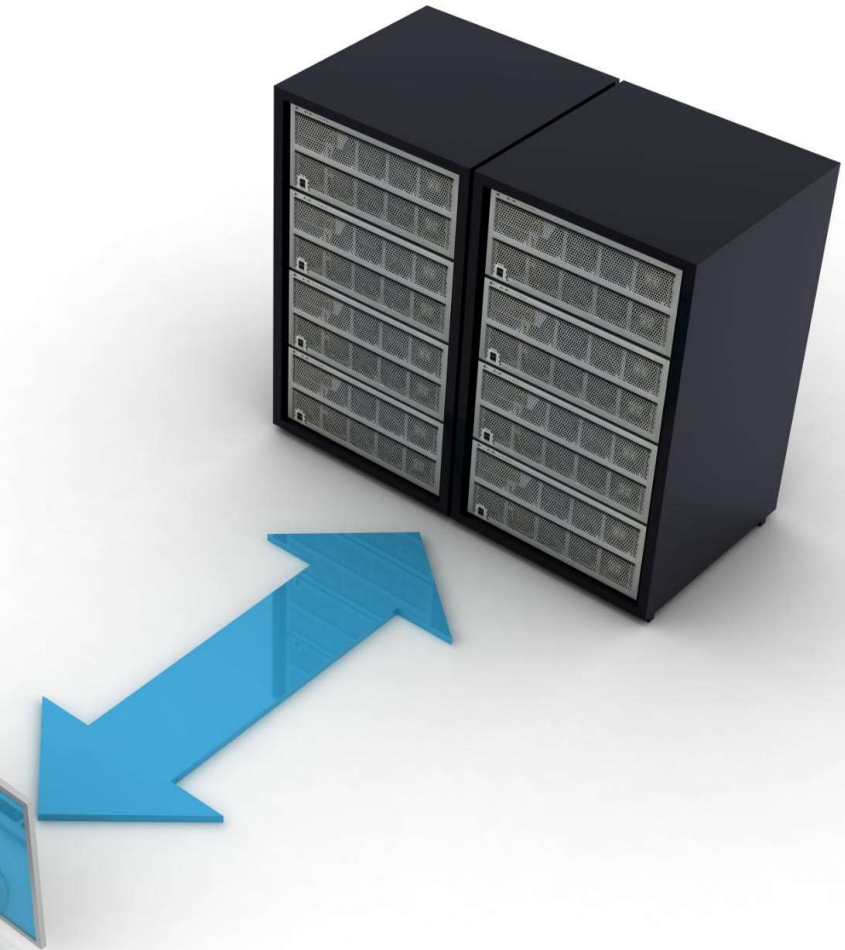
Microservices increase flexibility to scale category of services as per the processing need, supporting seamless resource allocation and easy system expansion.

Stateless Processing

Stateless processing simplifies scaling and recovery, making Realtime systems more resilient and easier to manage.

Container Autoscaling

Scaling containers through Kubernetes HPA (Horizontal Pod Autoscaler) lets applications respond to changing demand by increasing or decreasing container instances automatically.



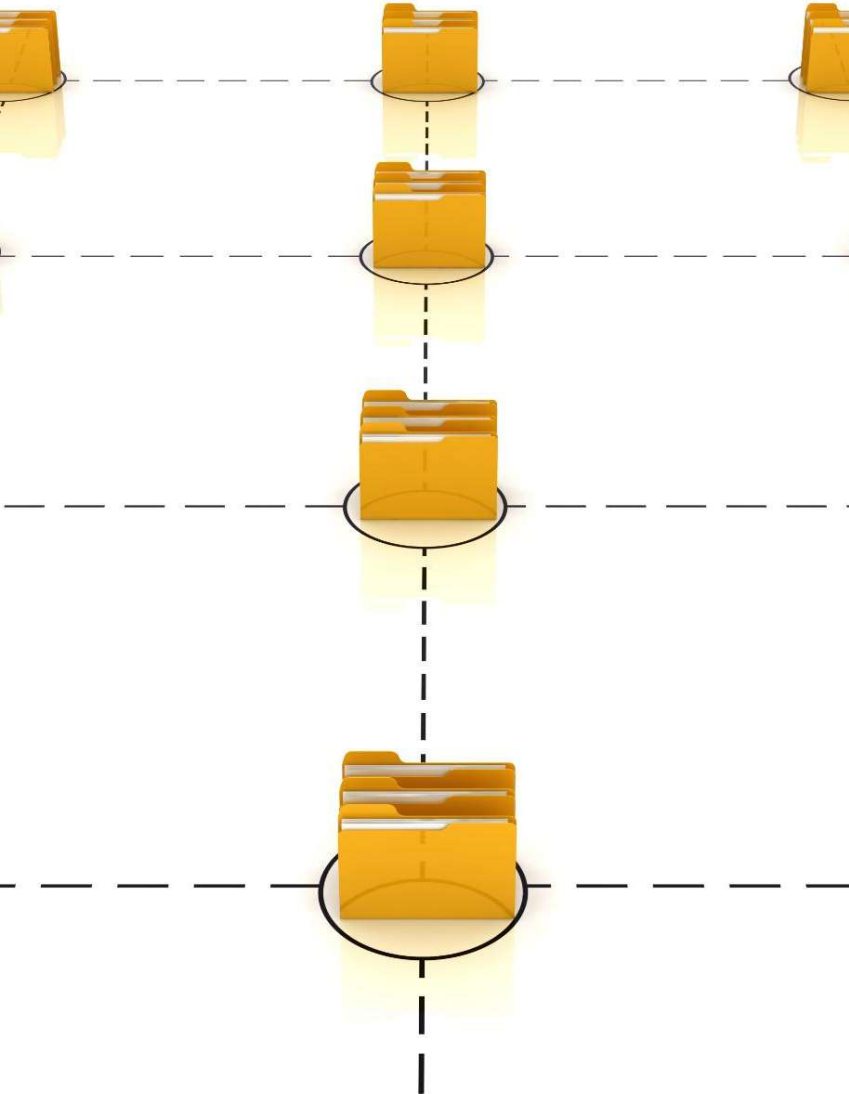
Near Realtime Processing Model (Events) – Scaling Design Techniques

Scaling Event Producers

Scaling Event Consumers

Event Consumer Autoscaling

kubernetes HPA based on CPU/Memory usage/lag /throughput.



Kafka Event Partitioning

Parallel Processing and Scalability

Event Partitioning enables data to be processed in parallel, supporting higher throughput and easier system scaling.

Order and Consistency

Messages with the same key always go to the same partition, ensuring order and data consistency within that partition.

Efficient Consumer Processing

Partitioning balances the load among consumers, allowing efficient and distributed data processing.



Bulk Processing Model (Batch) Scaling

Simultaneous Batch Execution

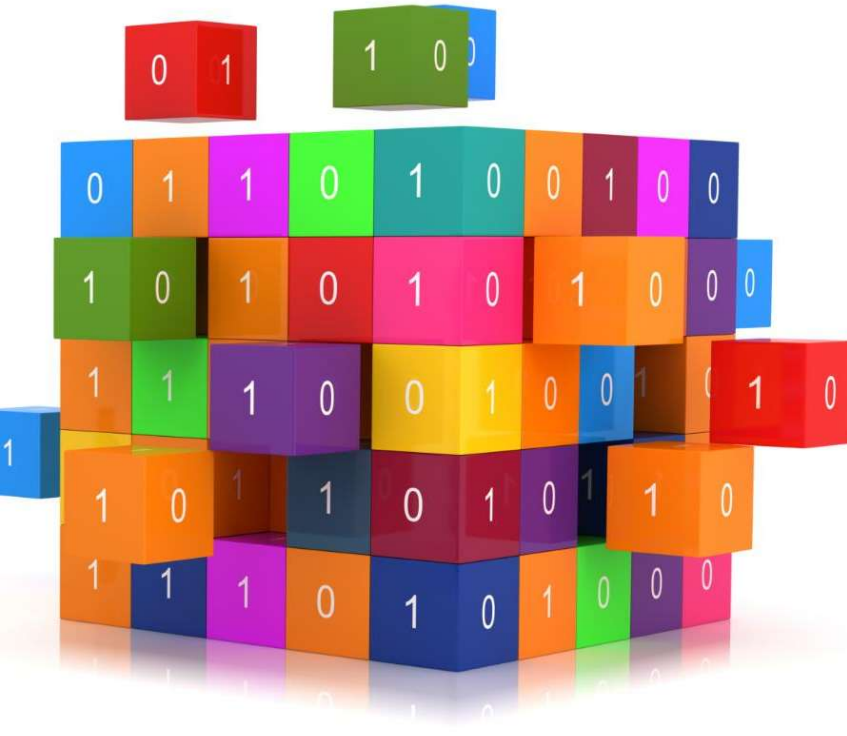
Multiple data batches are executed at the same time, leading to higher system throughput and faster results.

Efficient Data Chunking

Large datasets are divided into smaller, manageable chunks that can be processed in parallel for efficiency.

Resource Optimization and Scalability

This design makes optimal use of computing resources and enables efficient scaling in big data and machine learning.



Autoscaling Overview

Automatic Scaling

Container elasticity allows applications to scale up or down automatically depending on real-time demand, ensuring responsiveness.

Resource and Cost Efficiency

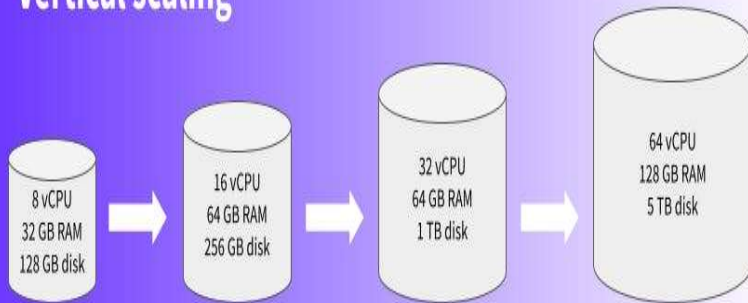
Elastic scaling ensures optimal use of resources and cost savings by matching resources to workload requirements.

Role of Orchestration Platforms

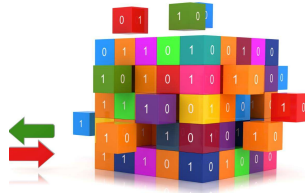
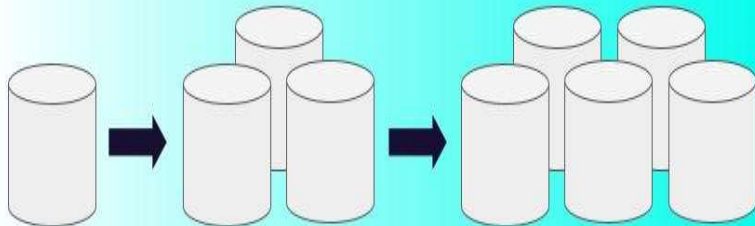
Platforms like Kubernetes manage container scaling dynamically, adjusting running containers in response to workload changes for high availability.

Database Scaling

Vertical scaling



Horizontal scaling



Database Scaling

Database scaling increases system capacity to manage larger data volumes or more throughput access needs



Vertical Scaling Approach

Vertical scaling enhances a single server by adding resources, boosting performance but facing hardware limitations.

Partitioning technique is used for segmenting Large Tables on a Single Server



Horizontal Scaling Method

Horizontal scaling uses multiple servers to distribute data, improving reliability and scalability with **sharding**



Database Sharding

What is Sharding?

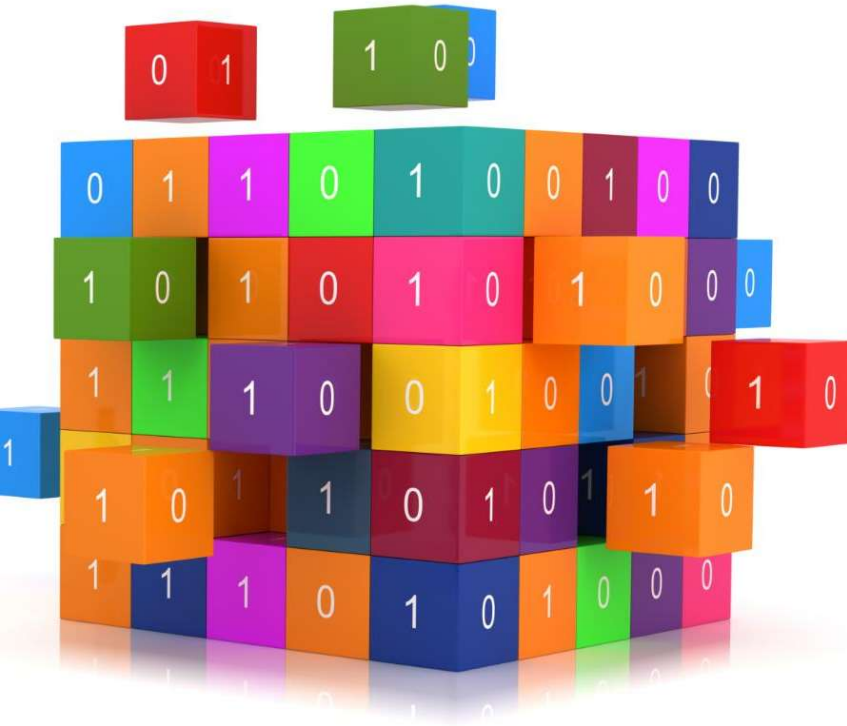
Sharding splits a large database into smaller, separate pieces called shards, making management easier and more efficient.

Improved Scalability and Performance

Sharding enhances scalability and performance by distributing data, reducing bottlenecks and handling larger volumes smoothly.

Horizontal Scalable Databases

CockroachDB and Citus (Postgres Extension) are some Databases that have been designed for Horizontal Scaling.



Database Partitioning

What is Database Partitioning

Partitioning divides large tables into smaller pieces, making data easier to handle and manage efficiently.

Enhanced Query Performance

Only relevant data segments are accessed, which speeds up queries and boosts overall database performance.

Efficient Maintenance

Backups, restores, and data purges can target partitions without disrupting the whole database, streamlining maintenance tasks.

Sharding vs Partitioning

Database Sharding

Sharding splits data across multiple servers, boosting scalability by distributing workload and enabling horizontal scaling.

Database Partitioning

Partitioning divides data within a single database into segments, making management easier and improving query performance.

Key Differences

Sharding spreads data across servers for horizontal scaling, while partitioning segments data within one server for efficiency.



Performance



Designing High Performance Core banking Platform

Rapid Transaction Processing

Modern core banking platforms process transactions swiftly (scaling to Tens of Thousands per second on Tens of Millions of Accounts), delivering seamless and efficient customer service experiences every day.

Performance Measures (In the Context of Core Banking)

- API's – Tiered API Response times (95th Percentile) e.g.
 - High Volume Financial Transactions and Inquires – e.g. Digital Deposits and Withdrawals, Account / Transaction, Balance Inquiries < 150ms
 - Low Volume but Large Processing Services – e.g. Account Open, Account Closure Services < 1 Sec
 - All Others < 500 ms
- Day End Completion – e.g.
 - Critical Day End Process – 2 Hours
 - Complete Day End Processes – 4 Hours



Caching Overview

Purpose of Caching

Caching temporarily stores data to enable faster access, reducing both server load and overall application response times.

Improved Application Performance

By serving frequently requested information from memory caching minimizes latency and eliminates the need for repeated data fetching from disk.

Types of Caching Strategies

Distributed caches and Transaction Level In Memory Thread Safe Cache each have unique advantages and are suited to different requirements and environments.

Distributed Caching

Boosts Speed and Scalability

Distributed caching stores data across servers to accelerate application performance and support scalable growth in demand.

Reduces Database Load

By retrieving data from memory, distributed caching eases the burden on central databases and improves system efficiency.

Ensures High Availability

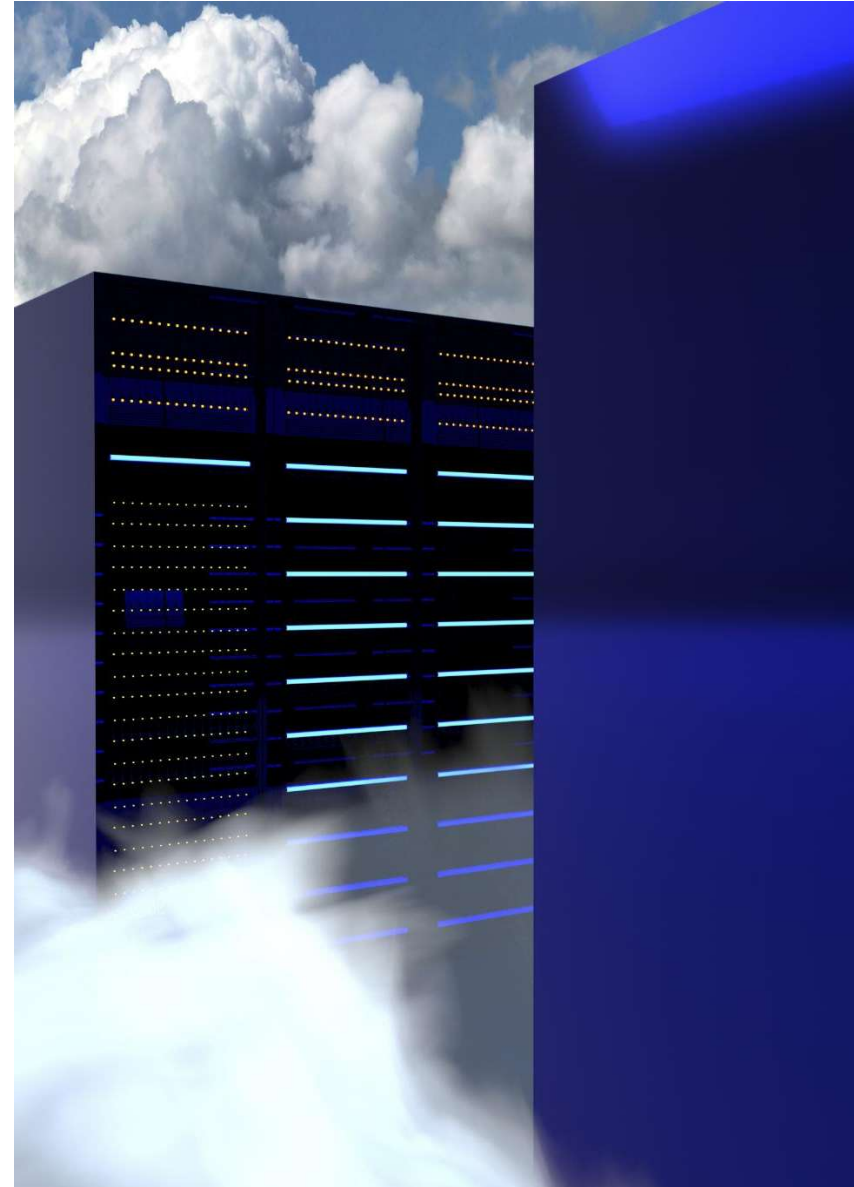
Distributed caching maintains fault tolerance and reliable data access, even in high-traffic environments.

Technologies

Distributed Caching Technologies like Infinispan, Redis enable horizontal scaling and ensure high availability for large applications.

Type of Data Cached for Core banking Platform

Static and Semi Static Data: Product and Pricing Configuration, Currency ISO Information, Bank Level Configuration ...



Database Indexes



Pointer-Based Navigation

Indexes act as pointers that guide queries directly to needed data, minimizing the need for full table scans.

Index Structure

A table index is a data structure consisting of set of fields on the Table is like a book's index mapping keywords to specific locations.

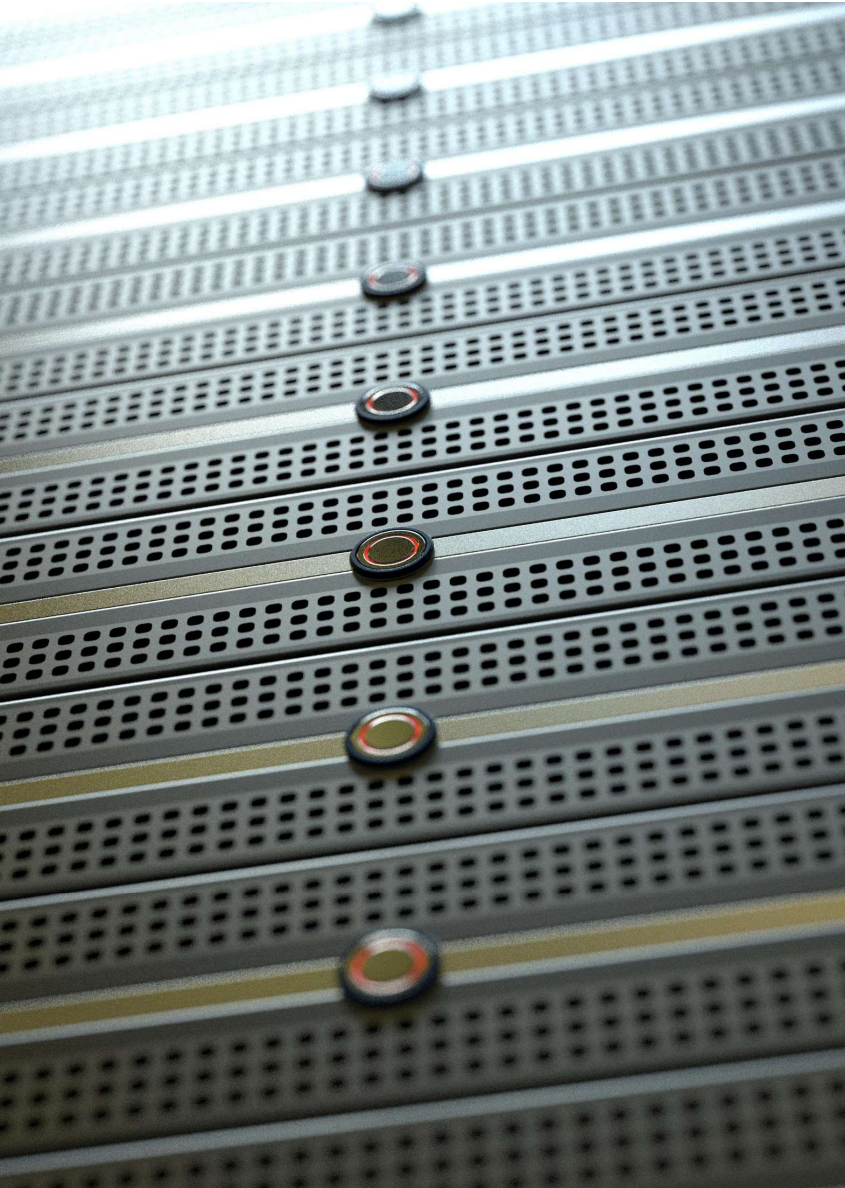
Improved Query Performance

Indexes enable faster data retrieval by allowing the database to quickly locate specific rows, boosting query speed significantly.

Indexes are crucial for large databases, making searches, queries and sorting much more efficient

Balancing Efficiency and Maintenance

Well-designed indexes improve efficiency, but too many can slow down inserts, updates, and deletes, requiring careful consideration.



Processing Optimization Techniques

Atomic Transaction Level “Cache”

- Java “Contexts and Dependency Injection (CDI)” introduces the concept of “contexts” which define the lifecycle and scope of managed beans.
- Container handles the creation and destruction of bean instances based on their defined scope.
- “Request Scoped” option provides Atomic Transaction Level “Cache” where data can be accessed within the specific Transaction processing flow without having to read from Database Tables.
- This approach enables faster response times while maintaining reliable data integrity, even during high traffic.

Type of Data Cached for Core banking Platform

Account, Balance, Transaction Data: Account Level Information, Balance Information, Transaction Data such as GL Postings



Processing Optimization Techniques

Parallel Execution

- Processing Steps where possible should be assessed if can be executed in Parallel rather than sequentially to reduce the response times e.g.
 - When Inquiring an Object (e.g. Account Inquiry) where multiple aggregates (e.g. Account Basic Details, Balances, Restrictions etc..) need to be returned, can the data access for each of these aggregates be initiated in Parallel if there is no dependency between them

Reducing Number of Database Inserts/Updates

- Database access reduction reduces the processing time, while Caching techniques mentioned earlier are to be used for Inquiries, even for Inserts/Updates below techniques can be considered:
 - Inserting Multiple Rows in a Table one time rather than Inserting each Row Separately
 - One Row having multiple similar Data Elements in a JSON Structure (e.g. All Balance Types for a Given Period are put in a Type/Value Pair in a Table Column and Inserted / Updated as One Row)

Performance Benchmarks

Measuring Software Efficiency

Performance benchmarks use standard tests to measure and compare the efficiency of different software systems objectively.

Identifying Bottlenecks

Benchmarks help pinpoint bottlenecks and areas for optimization to enhance software performance and responsiveness.

Capacity Sizing Data

Benchmarking provides objective data to build Capacity Sizing Calculators to Size hardware needed for Implementing a Core banking Platform

Performance Benchmark Centre's

Benchmark centers offer controlled environments for accurate testing and comparison of products or technologies, ensuring dependable results for organizations.

These centers help organizations find best practices, improve efficiency, and foster innovation through rigorous benchmarking and analysis





Performance Benchmarks – Test Types

Load Testing

Load testing measures how software behaves under expected workloads, helping identify bottlenecks and optimize resource usage.

Stress Testing

Stress testing pushes the system to extreme limits to uncover potential breaking points and ensure stability during peak conditions.

Resource and Data Scalability Testing

Scalability testing assesses how well software copes with increased Resources or data, confirming consistent response times as demand rises.

Longevity Testing

Longevity testing evaluates system behavior under sustained workloads for a large number of hours, sometimes lasting days to reveal hidden issues.

This test uncovers memory leaks, resource exhaustion and other stability concerns that only emerge over long durations.

Resiliency

Resilient Architectures

Resiliency has to be built at:

- Application Level
- Database Level
- Infrastructure Level

Note: Infrastructure Resiliency Like Network Resiliency, Hardware Resiliency is not covered in this course

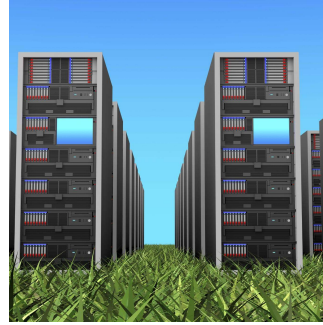
Resiliency Measures (In the Context of Core Banking)

Uptime Guarantee: System Availability

- e.g. 99.9% availability

Recovery Time: Recovery time from Outage

- RTO – Recovery Time Objective
- RPO – Recovery Point Objective



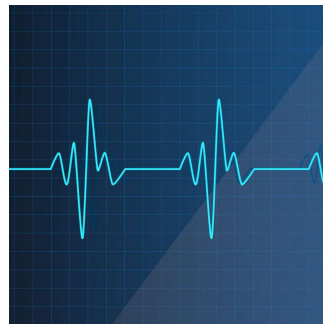
Withstanding and Recovering from Failures

Resilient systems are designed to tolerate failures and restore normal operation quickly, minimizing service disruption for users.



Redundancy and Distributed Components

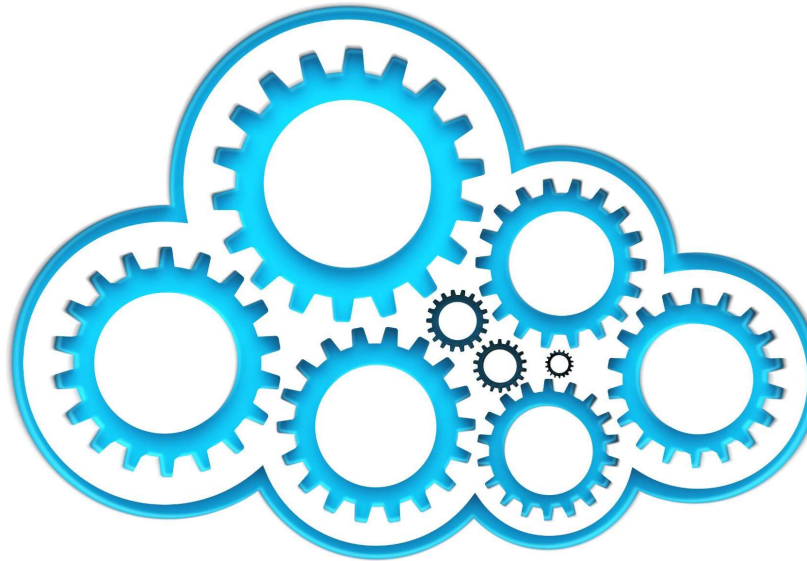
Redundant and distributed system components ensure that no single point of failure causes prolonged downtime, increasing reliability.



Testing and Monitoring

Regular system testing and monitoring enable prompt detection of issues and seamless service restoration, supporting consistent reliability.

Application Resiliency



Robust Error Handling

Strong error handling including Null Pointer Checks helps the application recover from unexpected events and prevents crashes or data loss.

Detecting Memory Leaks

Memory leaks happen when applications do not release unused memory, causing slowdowns and stability issues.

Profiling tools like Valgrind or IDE analysers help developers monitor and catch leaks efficiently.

Graceful Degradation

Graceful degradation keeps core functions operational even during system issues, improving user experience and reliability.

Resiliency Testing

Regularly resiliency testing using simulated failures such as chaos engineering to identify weaknesses and validate strategies.



Application Resiliency - Container Health Probes

Liveness Probes

Liveness probes check if a container is running. If the probe fails, Kubernetes restarts the container to maintain uptime.

Readiness Probes

Readiness probes show when a container is ready for traffic, ensuring requests go only to operational containers.

Enhanced Reliability

Health probes improve reliability and reduce downtime in containerized environments by automating recovery and traffic control.

Database Resiliency

Data Distribution Across Nodes / Shards

Distributing data across nodes / shards ensures high availability and performance, even if part of the system experiences issues.

Replication and Failover

Replication and failover mechanisms maintain database availability even during technical failures, ensuring uninterrupted business operations.

Backup and Recovery Strategies

Regular backups and robust recovery plans enable quick restoration of data after unexpected incidents or system failures.



Blue Green Deployment Model

Dual Production Environments

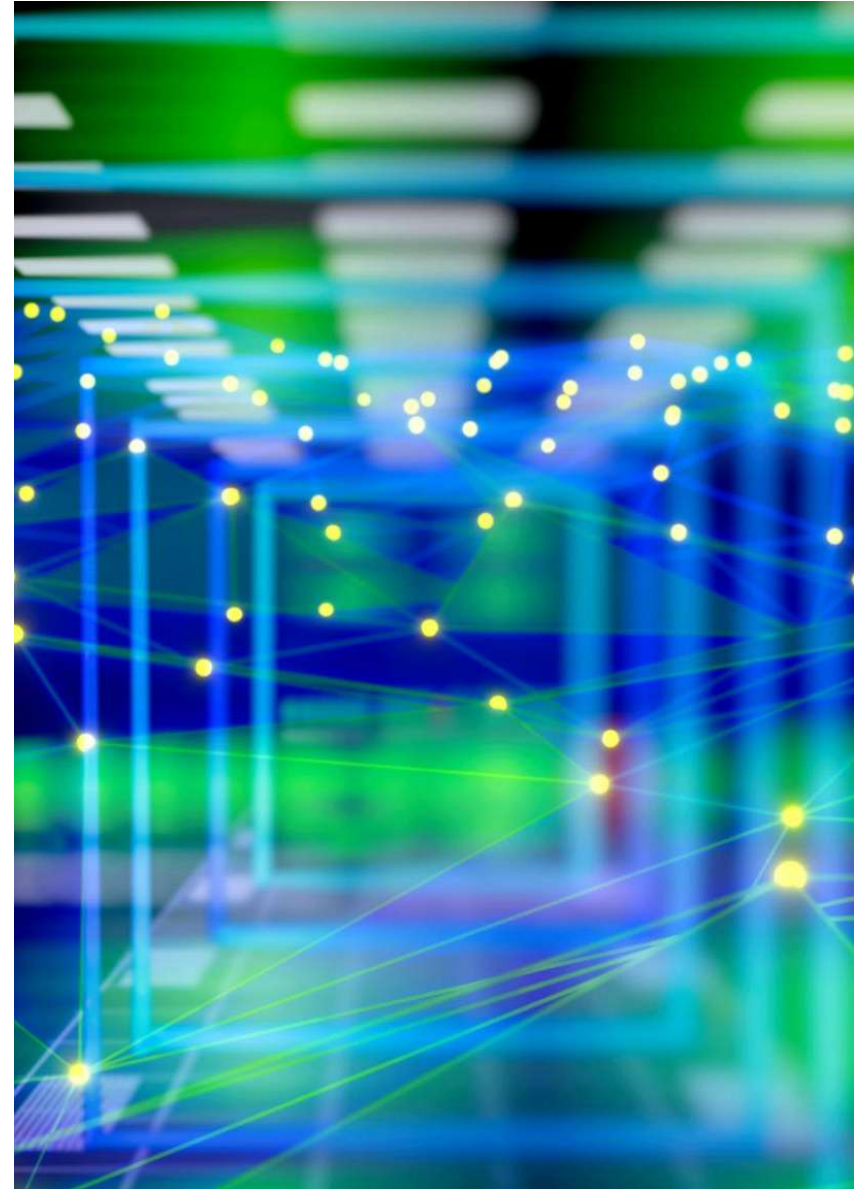
The model uses two identical environments, blue and green, reducing downtime and risk during software releases.

Seamless Traffic Switching

After successful testing, user traffic is directed from the blue environment to the green for seamless updates.

Easy Rollback Capability

If issues arise, reverting to the previous environment is quick and safe, ensuring reliable deployments.



Designing for Blue-Green Deployment

Backward-Compatible Database Schema Updates

Schema changes must support both blue and green deployments, ensuring database compatibility and service continuity.

Feature Toggles and Phased Changes

Feature toggles with phased data changes help maintain uninterrupted service and reduce deployment risks.

Decoupled Application Components

Designing applications with decoupled components allows for independent version management and easier deployment processes.

